Table of Contents

5.1 Interface 1

5.2 Abstract class 4

5.3 Final class 6

5.4 Test Your Knowledge 9

5.5 Summary or Take Away Points 9

5.6 References 10

5.1 Interface

An interface is a collection of abstract methods. A class implements an interface, thereby inheriting the abstract methods of the interface. In java interface is nothing but the collection of methods with empty implementations and constants variables (variables with static and final declarations). All the methods in an interface are "public and abstract" by default. Since interfaces are abstract in nature they cannot be directly instantiated. To define the methods of an interface the keyword "implements" is used.

An interface is not a class. Writing an interface is similar to writing a class, but they are two different concepts. A class describes the attributes and behaviors of an object. An interface contains behaviors that a class implements. Unless the class that implements the interface is abstract, all the methods of the interface need to be defined in the class.

Example:-

```
public interface Shape {

    public static final int LENGTH = 10;

    public abstract double area();

}
```

```java
public class Rectangle implements Shape {

    public double area() {

        return 200.316;
    }

}
```

```java
public class TestDemo {

    public static void main(String[] args) {

        int length = Shape.LENGTH;
        Shape s = new Rectangle();
        double area = s.area();

        System.out.println("length :" + length);
        System.out.println("area :" + area);

    }

}
```

Output:-

```
length :10
area :200.316
```

Rule's to be followed to use Interface:-

- If a class wants to make use an interface it should make use of it using keyword implements.
- A class can make use of any number of interfaces by separating the interface names with comma (,).
- If an interface is implemented by a class that class should override/implement all the methods in that Interface.
- An interface can be used by any number of classes.
- An interface can extend other interfaces.

- An interface can have variables but they should be final.
- An interface can have static and final variables then the class can make use of them with the help of interface name.

An Interface is similar to a class in the following ways:-

- An interface can contain any number of methods.
- An interface is written in a file with a .java extension, with the name of the interface matching the name of the file.
- The byte code of an interface appears in a .class file.
- Interfaces appear in packages, and their corresponding byte code file must be in a directory structure that matches the package name.

An Interface is different from a class in several ways, including:-

- We cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class; it is implemented by a class.
- An interface can extend multiple interfaces.

Rules to implement Interfaces are:-

- A class can implement more than one interface at a time.
- A class can extend only one class, but implement many interfaces.
- An interface can extend another interface, similarly to the way that a class can extend another class.

Extending Interfaces:-

The extends keyword is used to extend an interface, and the child interface inherits the methods of the parent interface.

Example:-

```
public interface Person extends Cloneable {

}
```

here **Cloneable** is the predefined interface available in java.lang package.

Extending Multiple Interfaces:-

A Java class can only extend one parent class. Multiple inheritance is not allowed. Interfaces are not classes, however, and an interface can extend more than one parent interface. The extends keyword is used once, and the parent interfaces are declared in a comma-separated list.

Example:-

```
import java.io.Serializable;

public interface Person extends Serializable, Cloneable {

}
```

here **Cloneable** is the predefined interface available in java.lang package. And **Serializable** also interface available in java.io package. These two interfaces are called **Marker or Tagged** interfaces they doesn't have any methods and variables.

5.2 Abstract class

An abstract class is a class that is declared abstract. It may or may not include abstract methods. Abstract classes cannot be instantiated, but they can be sub classed. An abstract method is a method that is declared without an implementation (without braces, and followed by a semicolon). If a class includes abstract methods, then the class itself must be declared abstract. When an abstract class is sub classed, the subclass usually provides implementations for all of the abstract methods in its parent class. However, if it does not, then the subclass must also be declared abstract. An abstract class can have data member, abstract method, method body, constructor and even main() method. If there is any abstract method in a class, that class must be abstract. If you are extending any abstract class that has abstract method, you must either provide the implementation of the method or make this class abstract.

Some important difference between Interface and Abstract classes are:-

Features	**Interface**	**Abstract Class**
Methods	An interface contains all the methods with empty implementation.	An abstract class may or may not have abstract methods but a class that is declared abstract.

Variables	The variables in interfaces are final and static.	Abstract classes may contain both instance as well as static variables.
Multiple Inheritance	In Java multiple inheritance is achieved by using the interface (by implementing more than one interface at a time)	Abstract classes do not provide this functionality.
Additional Functions	If we add a method to an interface then we will have to implement this interface by any class...	In Abstract classes we can add a method with default implementation and then we can use it by extending the abstract class

Example:-

```
abstract class Bike {
    abstract void run();

    void changeGear() {
        System.out.println("gear changed");
    }
}
class Honda extends Bike {
    void run() {
        System.out.println("running safely..");
    }

public class TestDemo {

    public static void main(String[] args) {

        Bike b=new Honda();
        b.run();

    }

}
```

```
Problems  @ Javadoc  Declaration  Servers  Console
<terminated> TestDemo [Java Application] C:\Program Files\Java\jre7\bin\jav
running safely
```

5.3 Final class

The **final** keyword in java is used to restrict the user. The final keyword can be used in many context. Final can be:

- Variable
- Method
- Class

Final Variable:-

If you make any variable as final, We cannot change the value of final variable(It will be constant).There is a final variable speed limit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

Example:-

```
public class Bike {

    public final int speedLimit = 20;

    public int run() {
        speedLimit = 40;
        return speedLimit;
    }

}
```

```
public class TestDemo {

    public static void main(String[] args) {

        Bike b = new Bike();
        b.run();

    }

}
```

Problems | Javadoc | Declaration | Servers | Console

```
<terminated> TestDemo [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe (Jun 5, 2014 12:36:07 PM)
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        The final field Bike.speedLimit cannot be assigned
```

Final method:-

If We make any method as final, you cannot override it.

Example:-

```
public class Bike {

    public final void run() {

        System.out.println("Inside Bike run");
    }

}
```

```
public class Honda extends Bike {

    public void run() {
        System.out.println("Inside Honda run");

    }

}
```

```
public class TestDemo {

    public static void main(String[] args) {

        Bike b = new Honda();
        b.run();

    }

}
```

Problems | Javadoc | Declaration | Servers | Console

```
<terminated> TestDemo [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe (Jun 5, 2014 12:28:56 PM)
Exception in thread "main" java.lang.VerifyError: class Honda overrides final method run.()V
```

Final Class:-

If We make any class as final, you cannot extend it.

```
public final class Bike {

    public void run() {
        System.out.println("Inside Bike run");
    }

}

public class Honda extends Bike {

    public void run() {
        System.out.println("Inside Honda run");

    }

}
```

```
public class TestDemo {

    public static void main(String[] args) {

        Bike b = new Honda();
        b.run();

    }

}
```

Problems | @ Javadoc | Declaration | Servers | Console

```
<terminated> TestDemo [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe (Jun 5, 2014 12:42:10 PM)
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        Type mismatch: cannot convert from Honda to Bike
```

5.4 Test Your Knowledge

1. Which of these keyword must be used to inherit a class?
   a) super
   b) this
   c) extent
   d) extends

2. Which of these is correct way of inheriting class A by class B?
   a) class B + class A {}
   b) class B inherits class A {}
   c) class B extends A {}
   d) class B extends class A {}

3. final class can be inherited True/False

4. It is possible to create Object for Final class True/False

5. abstract key word is mandatory for abstract class? True/False

6. abstract key word is mandatory for abstract method inside abstract class? True/False

7. It is possible to create abstract class with out any abstract methods? True/False

5.5 Summary or Take Away Points

- An interface will have only abstract methods (Methods without implementation)
- The variables in interfaces are declared as final and static.
- In Java multiple inheritance is achieved by using the interface (by implementing more than one interface at a time).
- An interface is used by implementing in a class. While implementing in an interface, all the methods of the interface will be defined in the implementation class. Else the implementation class will become an abstract class.
- Abstract classes may contain both instance as well as static variables.
- Final class means the class cannot extended.
- Final variable means its a constant and its value can not be changed.
- Final method means the method cannot not be overridden.

5.6 References

http://www.tutorialspoint.com/java/

http://www.roseindia.net/java/